🤯 Codificación en C (Clases 1, 2 y 3)

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano

📓 Clase 1 — Introducción al Lenguaje C

🎯 Objetivos

- Aprender de dónde viene C
- Conocer sus características principales
- Comprender cómo se procesa y ejecuta
- Escribir nuestro primer programa en C

🛠️ Herramientas

OnlineGDB es un entorno de desarrollo integrado (IDE) en la nube que permite editar, compilar y ejecutar programas en C/C++ con un solo clic.
👉 https://www.onlinegdb.com/

🏛️ Historia y Filosofía de C

- Creado por **Dennis Ritchie** en 1972 en los **Laboratorios Bell**.
- Diseñado originalmente para **UNIX**.
- Lenguaje de propósito general y alta eficiencia.
- Cada elemento se traduce en pocas instrucciones de máquina.
- Lenguaje de **nivel medio** (entre alto y bajo nivel).

> 👥 *Ken Thompson (sentado) y Dennis Ritchie fueron pioneros en C y UNIX.*

💻 Usos de C Hoy

- Sistemas operativos (Unix, Linux, partes de Windows).
- Compiladores e intérpretes.
- Lenguajes modernos basados en C: **C++, C#, Java**.
- Sistemas embebidos y controladores.
- Motores de videojuegos, bases de datos, aplicaciones de tiempo real.

⚙️ Características Fundamentales

- **Eficiente** → Control preciso del hardware.
- **Portable** → Corre en múltiples plataformas.
- **Flexible** → Distintas formas de resolver problemas.

- **Bajo nivel** → Acceso directo a memoria y bits.
- **Sintaxis simple** → Fácil de leer y aprender.
- **Estructurado** → Modular y ordenado.
- **Biblioteca estándar** → Funciones listas para usar.

🔄 Procesamiento del Código en C

1. **Codificación:** escribir el código fuente (`programa.c`).
2. **Preprocesamiento:** expansión de `#include`, `#define`, eliminación de comentarios → genera `.i`.
3. **Compilación y ensamblado:** traducción a `.s` y luego a `.o`.
4. **Enlazador (Linker):** une los `.o` y bibliotecas → ejecutable final `.exe`.

☁️ Ejecución Online

OnlineGDB simplifica todo el proceso:

- Escribir el código
- Hacer clic en **Run**
- Ver resultados instantáneos

🩹 Primer Programa en C

```c
#include <stdio.h>

int main() {
    printf("Hola Mundo!");
    return 0;
}
```

☑ ¡Nuestro primer programa en C!

📓 Clase 2 — Estructura y Tipos de Datos

🎯 Objetivos

- Aprender la estructura general de un programa en C
- Usar librerías
- Conocer constantes, variables y tipos de datos
- Manejar entrada y salida

📦 Estructura General

1. Inclusión de bibliotecas → `#include`
2. Definición de constantes → `#define`

3. Variables globales
4. Prototipos de funciones
5. Función `main()`
6. Definiciones de funciones

```c
// 1. INCLUSIÓN DE BIBLIOTECAS
#include <stdio.h>
#include <stdlib.h>

// 2. DEFINICIÓN DE CONSTANTES
#define PI 3.1416
#define MAX 100

// 3. DECLARACIÓN DE VARIABLES GLOBALES
int contador_global;
float temperatura_global;

// 4. PROTOTIPOS DE FUNCIONES (DECLARACIONES)
void funcion_ejemplo(int parametro);

// 5. FUNCIÓN PRINCIPAL
int main()
{
    // (Aquí irían las sentencias del programa)
    return 0;
}

// 6. DEFINICIÓN DE FUNCIONES (IMPLEMENTACIONES)
void funcion_ejemplo(int parametro)
{
    // Cuerpo de la función
}
```

Librerías

Librería	Función principal	Ejemplos
`stdio.h`	Entrada/salida	`printf()`, `scanf()`
`stdlib.h`	Funciones generales	`rand()`, `system()`
`math.h`	Operaciones matemáticas	`sqrt()`, `pow()`
`string.h`	Manejo de cadenas	`strlen()`, `strcpy()`
`time.h`	Manejo de tiempo	`time()`, `clock()`

Constantes

Son valores fijos que no cambian durante la ejecución del programa.

- Se definen una vez y se usan en todo el código.
- Mejoran la legibilidad y facilitan el mantenimiento.
- Por convención, se escriben en MAYÚSCULAS

◇ Con `#define`

- Directiva del preprocesador.
- Reemplaza el valor antes de compilar.
- No tiene tipo de dato.

```
#define PI 3.1416
#define MAX 100
```

Ejmplo:

```
#include <stdio.h>
#define MAX_INTENTOS 3

int main(){
    for(int i=0; i<MAX_INTENTOS; i++){
        printf("Intento %d\n", i+1);
    }
    return 0;
}
```

◇ Con `const`

- Una variable cuyo valor no puede cambiarse después de inicializarla.
- El compilador sí valida el tipo de dato.
- Generalmente más seguro y legible que #define.

Características:

- Puede ocupar memoria en tiempo de ejecución.
- El compilador ayuda a detectar errores de tipo.
- Buena práctica para constantes numéricas o de texto.

```
const float PI = 3.1416;
```

```
#include <stdio.h>

int main() {
    const int MAX_INTENTOS = 3;
```

```
    for(int i = 0; i < MAX_INTENTOS; i++) {
        printf("Intento %d\n", i + 1);
    }
    return 0;
}
```

Variables

¿Qué son las variables?

- Espacios de memoria reservados identificados con un nombre.
- Posee un tipo que determina qué clase de datos puede contener.
- Deben declararse antes de ser utilizadas.

¿Para qué sirven?

- Almacenar valores que pueden modificarse durante la ejecución.
- Permitir operaciones con datos dinámicos.
- Dar nombre significativo a los datos que maneja el programa.

¿ Como se declaran ?

- Sintaxis básica:

```
  tipo nombre_variable;
```

Ejmplo:

```
  // Declaración de variables básicas en C
int edad;        // Variable entera
float promedio;  // Variable de punto flotante
char inicial;    // Variable tipo carácter
unsigned int contador;  // Entero sin signo (solo valores positivos)
double distancia;  // Variable de punto flotante con doble precisión
```

Tipos de Datos

En C, cada variable debe declararse con un tipo de dato, el cual determina:

- Tamaño en memoria (en bits o bytes).
- Rango de valores que puede almacenar.
- Formato de impresión al mostrar su contenido con printf.

Tipo	Tamaño	Rango	Formato
`char`	8 bits	-128 a 127	`%c`
`int`	16 bits	-32768 a 32767	`%d`
`long`	32 bits	-2147483648 a 2147483647	`%ld`
`float`	32 bits	3.4E-38 a 3.4E+38	`%f`
`double`	64 bits	1.7E-308 a 1.7E+308	`%lf`

> Se puede usar `unsigned` para ampliar el rango positivo (ej: `unsigned int` → 0 a 65535).

Cuerpo del Programa

Estructura básica

- El punto de entrada es la función principal main().
- Esta función principal también tiene un tipo de dato de retorno
- Las llaves delimitan el bloque de código

```
comienzo()
{
    A = 5;
    B = 8;
    C = 1;
    suma = A + B + C;
    imprimir("La suma es: ", suma);
}
```

```c
#include <stdio.h>   // Inclusión de la librería estándar para entrada/salida

int main()
{
    // Declaración de variables
    int A = 5;
    int B = 8;
    int C = 1;
    int suma;

    // Operación de suma
    suma = A + B + C;

    // Salida del resultado
    printf("La suma es: %i\n", suma);

    return 0; // Fin del programa sin errores
}
```

⌨ Entrada y Salida

- **Salida:**

```
printf("Texto %d", variable);
```

- 🖨 Ejemplos de salida formateada en C

```
int edad = 25;
printf("Tengo %i años", edad);     // Salida: Tengo 25 años

float precio = 99.99;
printf("Precio: %.2f", precio);    // Salida: Precio: 99.99
```

- **Entrada:**

```
scanf("%d", &variable);
```

- 📥 Entrada de datos en C con `scanf`

```
int cantidad;
scanf("%d", &cantidad);       // Lee un entero

float temperatura;
scanf("%f", &temperatura);    // Lee un float

char inicial;
scanf("%c", &inicial);        // Lee un carácter
```

> El operador `&` obtiene la dirección de memoria donde guardar el valor.

📚 Bibliografía Recomendada

- *El lenguaje C: Manual de Referencia* — Balestrini & Di Stefano
- *Programación en C para principiantes* — Urrutia & Landa
- *Introducción a la programación en C* — Marzal & Gracia
- *GNU C Language Manual* — Richard Stallman

📘 Clase 3 — Estructuras de Control

🎯 Objetivos

- Conocer operadores en C
- Usar estructuras de control (`if`, `else`, `switch`)
- Comprender ciclos: `for`, `while`, `do while`

➕ Operadores Aritméticos

Operador	Descripción	Ejemplo
`=`	Asignación	`a = 1`
`+`	Suma	`a + b`
`-`	Resta	`a - b`
`*`	Multiplicación	`a * b`
`/`	División	`a / b`
`%`	Módulo	`a % b`

Incremento / Decremento

Operador	Ejemplo	Descripción
`++`	`a++` o `++a`	Incrementa
`--`	`a--` o `--a`	Decrementa

> `a++` incrementa *después* de usar el valor; `++a` incrementa *antes*.

⚖️ Operadores de Comparación

Operador	Descripción	Ejemplo
`==`	Igual a	`a == 1`
`!=`	Diferente de	`a != b`
`>`	Mayor que	`a > b`
`<`	Menor que	`a < b`
`>=`	Mayor o igual	`a >= b`
`<=`	Menor o igual	`a <= b`

🔗 Operadores Lógicos

Operador	Nombre	Ejemplo

Operador	Nombre	Ejemplo
`!`	NOT	`!a`
`&&`	AND	`a && b`
`II`	OR	`a II b`

🧮 Precedencia de Operadores

1. Unarios `++ --`
2. Multiplicación / División / Módulo `* / %`
3. Suma / Resta `+ -`
4. Asignación `=`

Asociatividad:

- Generalmente de izquierda a derecha.
- Excepto en unarios y asignación (derecha a izquierda).

🧠 Condicionales

◇ if / else

```
if (condición) {
    // código si es verdadero
} else {
    // código si es falso
}
```

> ❗ Error común: usar `=` en lugar de `==` para comparar.

◇ switch

- Se usa cuando una variable debe evaluarse contra múltiples valores posibles.
- Más claro y ordenado que muchos if/else if.
- break evita el "efecto cascada" (ejecutar más de un bloque).
- default asegura cubrir cualquier otro valor.
- Muy útil con menús y opciones de usuario.

```
switch (opcion) {
    case 1:
        // código
        break;
    case 2:
        // código
```

```
            break;
        default:
            // caso por defecto
            break;
    }
```

Estructuras Repetitivas

for (bucle exacto)

for - Estructura repetitiva exacta

- Se usa cuando sabemos cuántas veces se repetirá el bucle.
- Toda la configuración está en la cabecera:
    - Inicialización → valor inicial de la variable.
    - Condición → controla cuántas veces se repite.
    - Incremento/Decremento → modifica la variable de control.

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
}
```

Usado cuando se conoce el número de iteraciones.

while (0 a n veces)

while - Estructura repetitiva inexacta (0 a n)

- La condición se evalúa antes de entrar al bucle.
- Ejecuta el bloque mientras la condición sea verdadera.
- Puede ejecutarse 0 veces si la condición inicial es falsa.

```
while (condición) {
    // código
}
```

do while (1 a n veces)

do while - Estructura repetitiva inexacta (1 a n)

- La condición se evalúa al final del ciclo.
- Variante del while.
- El bloque se ejecuta al menos una vez, incluso si la condición es falsa desde el inicio.

```c
do {
    // código
} while (condición);
```

> Se ejecuta al menos una vez.
